

Opynio Widget Loader — v6.4.0

Carga perezosa SEO-safe + reservación de espacio + timers visibility-aware (con fixes de revisión independiente)

v6.0 →
v6.4.0

Por qué se hizo

El loader v6.0 inicializaba **todos** los widgets en `DOMContentLoaded`, sin importar si estaban dentro o fuera del viewport del host. Cada init disparaba 2 round-trips de red (Supabase + Google Translate) en paralelo al render del cliente. Resultado: degradación del LCP/INP del host y riesgo de penalización por Core Web Vitals.

Cambios aplicados (público: `public/widget.js`)

Componente	v6.0	v6.4.0
Trigger de inicialización	Todos a la vez en <code>DOMContentLoaded</code>	<code>IntersectionObserver</code> con <code>rootMargin: '200px 0px'</code> por widget
Defer post-LCP del host	—	<code>requestIdleCallback(timeout: 1500)</code> antes de cada init
Bypass para crawlers	Sólo render interno	Scheduler respeta <code>IS_BOT</code> → render inmediato sin observer
Reservación de espacio (CLS)	—	<code>min-height</code> por tipo aplicado antes de hidratar
Timers de carruseles	<code>setInterval</code> eternos	<code>visibilityAwareInterval</code> : pausa al salir de viewport y con <code>document.hidden</code> ; teardown completo en <code>stop()</code> (IO + listener + interval)
MutationObserver	<code>document.body subtree</code> , dispara siempre	Filtra <code>.opynio-widget</code> , debounce 50 ms; limpia recursos en <code>removedNodes</code> (SPAs)

Fixes post-revisión (v6.1 → v6.4.0)

v6.1.1 — leaks detectados por revisión independiente

- **Leak de listeners `visibilitychange`**: cada re-arm del ticker dejaba un listener pegado a `document`. `stop()` ahora hace `removeEventListener`.
- **Leak de `IntersectionObserver` internos**: cada re-arm creaba un IO nuevo sin desconectar el anterior. `stop()` ahora hace `io.disconnect()`.
- **Cleanup en SPAs**: el `MutationObserver` ahora itera `removedNodes` y libera `lazyObserver` + ticker asociado.
- **Race resuelto** entre IO y `visibilitychange`: `tick()` filtra, el listener llama `start()` incondicional.
- **Test page**: `document.hidden` restaurado con `delete`; checks rAF-throttled durante scroll.

v6.1.2 — endurecimiento adicional

- **BOT_REGEX ampliado**: añadidos `Mediapartners-Google` (AdSense), `Chrome-Lighthouse` (PageSpeed Insights), `TelegramBot` y `Pinterest`. Lighthouse en cliente ya no califica al host con un estado lazy a medio cargar.
- **reserveSpace respeta CSS externo**: lee `getComputedStyle(el).minHeight` antes de aplicar el default, así un host con `.opynio-widget { min-height: 200px }` en su CSS gana.
- **Chrome prerender**: `document.prerendering === true` fuerza render eager (sin observer) — el page activado al usuario ya está listo.

v6.2.0 — limpieza SEO definitiva

- **JSON-LD `AggregateRating` eliminado del host**: Google ignora el rich result self-serving para `LocalBusiness/Organization` desde 2019. La structured data canónica vive sólo en `web.opynio.com`.
- **Bot-path centralizado para los 9 widgets**: cuando `IS_BOT`, `initWidget` llama a `renderBotSafe()`. Bots ven sólo brand + estrellas + count + anchor canónico — nunca el contenido completo de las reseñas. Elimina duplicate content entre hosts.
- **`rel="noopener nofollow"` en los 10 anchors `target="_blank"`** y en el `window.open` del sidebar. Cierra tabnabbing y neutraliza el patrón de link scheme.

v6.3.0 — UX floating + cache de traducciones

- **Floating con botón cerrar:** × discreto en la esquina del pill. Cliente lo cierra → dismissal en `localStorage` con TTL 7 días; `scheduleWidget` lo respeta sin fetch.
- **data-position:** `bottom-left` (default), `bottom-right`, `top-left`, `top-right`. El host elige la esquina para no chocar con su cookie banner / chat.
- **Cache persistente de traducciones:** `translateReviews` respaldado en `localStorage` con TTL 7 días, flush debounced, fallback in-memory si la cuota se llena.
- **UI string** `close` en los 13 idiomas hardcoded — sin caer al fallback español en hosts internacionales.

v6.4.0 — Shadow DOM (aislamiento total)

- **attachShadow({mode: 'open'})** en cada widget humano. CSS del host nunca pisa al widget; CSS del widget nunca leakea al host. Fin de la guerra de especificidad.
- **Stylesheet compartido** via Constructable Stylesheets (`adoptedStyleSheets`) cuando está soportado: una sola `CSSStyleSheet` reutilizada por todos los widgets en lugar de N copias parseadas N veces. Fallback a `<style>` por shadow root.
- **Bot path en light DOM:** crawlers indexan el anchor canónico sin atravesar shadow trees. Browsers sin `attachShadow` también caen a light DOM (degradación segura).
- **!important eliminados del shadow scope:** dentro del shadow nada los necesita. En el CSS light (bot path + fallback) se mantienen como defensa.
- **Renderers con firma** (`root`, `el`, `business`, `reviews`, `s`): `root` es la raíz de render (wrapper en shadow o el host en light), `el` sigue accesible para `dataset` y eventos a nivel host.
- **Test page actualizado:** helpers `queryInside()/queryAllInside()` que atraviesan shadow roots; nuevo invariante "Shadow activo en humanos / light DOM en bots"; el test de pausa por `document.hidden` ahora se ejecuta en automático.

Veredicto post v6.4.0: el widget cumple la regla maestra "*neutral o positivo, nunca negativo*" en CWV, indexación, structured data, autoridad de enlace e **aislamiento CSS**. Es la primera versión en la que ningún tema del host (WordPress, Shopify, etc.) puede romper el render del widget. No quedan pendientes SEO bloqueantes ni higiene relevante.

Decisiones clave

rootMargin 200 px

Pre-renderiza el widget justo antes de entrar al viewport. El usuario ve contenido, no spinner. Recomendado por Google Search Central porque no depende de scroll.

Bypass de Googlebot

Google Search Central: "*Googlebot doesn't scroll, scroll events never fire*". `IS_BOT` salta el observer y renderiza directo, garantizando indexación.

requestIdleCallback timeout 1500 ms

El widget no compite con el LCP del host. Si el navegador no está idle a tiempo, el timeout fuerza el render. Fallback a `setTimeout`.

min-height por tipo

Cada widget reserva su espacio antes de pintar contenido (badge 90px → wall 560px). CLS aportado ≈ 0. Floating omitido (es position:fixed).

Método de validación

Página de test sin dependencias: `public/widget-test.html` — se sirve con `npm run dev` en `http://localhost:3000/widget-test.html`.

- Mockea `fetch` a `widget-proxy` y a Google Translate (corre offline).
- Monta 5 widgets a distintas alturas + 1 floating.
- Panel sticky con 10 invariantes evaluadas en vivo: lazy schedule, espacio reservado, eager-init guard, errores JS, CLS aportado, pausa por visibility, bypass de bot.
- Modo `?bot=1` → fuerza `navigator.userAgent` a Googlebot y valida bypass.
- Botón "Ocultar pestaña" → simula `document.hidden` 5 s y compara `transform` de carruseles.

Validación oficial pendiente (manual, post-deploy en staging): Google Search Console → URL Inspection → *Test Live URL* · Rich Results Test · PageSpeed Insights antes/después.

Pendientes

Ninguno bloqueante. Los cuatro hallazgos identificados en la auditoría inicial quedaron cerrados:

Versión	Cierra
v6.1.x	Lazy loading SEO-safe + visibility-aware timers + leaks fixed
v6.2.0	JSON-LD self-serving fuera del host · <code>rel="noopener nofollow"</code> · bot-path en los 9 widgets
v6.3.0	UX del floating (cerrar + posición + persistencia) · cache persistente de traducciones
v6.4.0	Shadow DOM para aislamiento total de CSS + Constructable Stylesheets compartidos

Archivos modificados / creados

- **Modificado:** `public/widget.js` — scheduler lazy + visibility-aware + bot-path centralizado + JSON-LD eliminado + rel en anchors + UX floating + cache traducciones + Shadow DOM (v6.0 → v6.4.0)
- **Nuevo:** `public/widget-test.html` — página de test con mock de fetch, panel de 11 invariantes en vivo, y piercing de shadow roots
- **Nuevo:** `docs/11-WIDGET-LAZY-LOADING-V6.1.md` — documentación técnica completa del cambio
- **Nuevo:** `docs/12-WIDGET-SHADOW-DOM-PLAN.md` — plan original de Shadow DOM (ahora ejecutado en v6.4.0)